

Universidade Federal de Ouro Preto - UFOP
Instituto de Ciências Exatas e Biológicas - ICEB
Departamento de Computação - DECOM
Ciência da Computação

Trabalho Prático II

PCC104 - Projeto e Análise de Algoritmos

Fernando dos Santos Alves Fernandes
Professor: Pedro Henrique Lopes Silva

Ouro Preto
17 de junho de 2024

Sumário

1	Introdução	1
1.1	O Problema do Clique Máximo	1
1.2	O Problema do Mínimo Número de Cliques	1
1.3	Os Algoritmos <i>Backtracking</i> e <i>Branch&Bound</i>	1
1.4	O Algoritmo de <i>Bron-Kerbosch</i>	2
1.5	Visão geral do ambiente de desenvolvimento e testes	3
1.5.1	Ambiente de desenvolvimento e ferramentas utilizadas	3
1.5.2	Instruções de compilação e execução	3
2	Implementação	4
2.1	Implementações para o Problema do Clique Máximo	4
2.2	Implementações para o Problema do Mínimo Número de Cliques	9
3	Estudo de Complexidade	13
3.1	Análise de complexidade para o Problema do Clique Máximo	13
3.2	Análise de complexidade para o Problema do Mínimo Número de Cliques	13
4	Testes	13
4.1	Testes para o Problema do Clique Máximo	14
4.2	Testes para o Problema do Mínimo Número de Cliques	15
5	Análises e Considerações	18

Lista de Tabelas

1	Exemplo de entrada e saída esperada para a Tarefa 1	14
2	Exemplo de entrada e saída para a Tarefa 2	15
3	Resultados dos testes para o problema do clique máximo.	18

Lista de Códigos Fonte

1	Pseudo-código - Abordagem <i>Backtracking</i> . Adaptado de [2]	1
2	Pseudo-código - Abordagem <i>Branch&Bound</i> . Adaptado de [2]	2
3	Pseudo-código do Algoritmo de Bron-Kerbosch para o Problema de Enumeração dos Cliques Maximais (Backtracking). Adaptado de [5]	2
4	Implementação da Tarefa 1 - Problema do Clique Máximo, usando <i>Backtracking</i> e <i>Branch&Bound</i>	4
5	Implementação da Tarefa 2 - Problema do Mínimo Número de Cliques, com <i>Backtracking</i>	9

1 Introdução

Neste segundo trabalho prático da disciplina de Projeto e Análise de Algoritmos, as tarefas estão relacionadas a lidar com dois problemas computacionais combinatórios: obter o *maior clique dentro de um grafo não-direcionado* e obter o *menor número de cliques dentro de um grafo não-direcionado*.

A *Tarefa 1* consistiu em implementar os algoritmos de *Backtracking* e *Branch&Bound* para resolver o primeiro problema, enquanto a *Tarefa 2* consistiu em utilizar as mesmas abordagens para resolver o segundo problema. O custo de tempo dos dois algoritmos pode ser de $O(2^n)$, contudo, é possível reduzir os espaços de busca por meio de "podas", que evitam considerar soluções "inviáveis".

1.1 O Problema do Clique Máximo

Um clique em um grafo não-direcionado $G = (V, E)$ é um subconjunto $V' \subseteq V$ de vértices, em que cada par $\{u, v\}$, com $u \in V'$ e $v \in V'$, é uma aresta em E . Em outras palavras, um clique induz um subgrafo completo em G . O tamanho do clique é o número de vértices que ele contém. O *problema do clique máximo* é um problema de otimização, cujo objetivo é em encontrar o clique de tamanho máximo em um grafo G [1].

1.2 O Problema do Mínimo Número de Cliques

O problema dos subconjuntos consiste na minimização do número de cliques dentro de um grafo não-direcionado. O problema de encontrar o número mínimo de cliques em um grafo G está intimamente relacionado ao problema de particionar os vértices de G em subconjuntos, em que cada subconjunto forma um clique. Esse problema é também conhecido como o *problema de cobertura por cliques* (*clique over problem*).

1.3 Os Algoritmos *Backtracking* e *Branch&Bound*

Dentre as possíveis abordagens para resolver os dois problemas mencionados, encontram-se os algoritmos de busca exaustiva com "poda": *Backtracking* e *Branch&Bound*.

O algoritmo de *Backtracking* é um método recursivo para construir, um passo de cada vez, soluções factíveis para problemas de otimização combinatória. É um algoritmo de *busca exaustiva*, uma vez que todas as soluções possíveis são consideradas. Métodos de poda podem ser usados para evitar considerar soluções menos promissoras [3].

O algoritmo de *Backtracking* trabalha explorando recursivamente todas as possíveis soluções para um problema. Começa escolhendo uma solução inicial, então explora todas as possíveis extensões daquela solução. Se uma extensão leva a uma solução, o algoritmo retorna essa solução. Se uma extensão não leva a uma solução, o algoritmo retrocede para uma solução anterior e tenta uma extensão diferente. O procedimento se repete até que uma solução seja encontrada ou até que todo o espaço de busca tenha sido explorado. O pseudo-código genérico para o *Backtracking* pode ser visto em Código 1.

```
1 Start with some problem P0
2 1 Let S = {P0}, the set of active subproblems
3 2 Repeat while S is non-empty:
4 3   choose a subproblem P in S and remove it from S
5 4   expland it into smaller subproblems P1, P2, ..., Pk
6 5   for each Pi:
7 6     If test(Pi) succeeds: halt and annouce this solution
8 7     If test(Pi) fails: discard Pi
9 8     Otherwise: add Pi to S
10 9 Announce that there is no solution
```

Código 1: Pseudo-código - Abordagem *Backtracking*. Adaptado de [2]

O algoritmo de *Branch&Bound* é um método usado em problemas de otimização para, sistematicamente, encontrar a melhor solução. O algoritmo trabalha dividindo o problema em subproblemas menores, ou ramificações, eliminando certos ramos com base em limites ou restrições em relação à solução ótima. Esse processo continua até que a melhor solução seja encontrada ou todos os ramos

tenham sido explorados. O pseudo-código genérico para o *Branch&Bound* pode ser visto em Código 2.

```

1 Start with some problem P0
2 1 Let S = {P0}, the set of active subproblems
3 2 bestSoFar = INF
4 2 Repeat while S is non-empty:
5 3     choose a subproblem P in S and remove it from S
6 4     expland it into smaller subproblems P1, P2, ..., Pk
7 5     for each Pi:
8 6         If Pi is a complete solution: update bestSoFar
9 7         Else If lowerBound(Pi) < bestsofar: add Pi to S
10 9 return bestSoFar

```

Código 2: Pseudo-código - Abordagem *Branch&Bound*. Adaptado de [2]

Uma das principais diferenças entre os algoritmos *Backtracking* e *Branch&Bound* está na estratégia de busca utilizada. Enquanto o primeiro realiza uma busca em profundidade, o segundo utiliza a busca em largura. O *Branch&Bound* também mantém algum critério para a escolha dos vértices que ainda não foram explorados, além de definir limites (superiores ou inferiores) para avaliar se vale a pena continuar explorando determinado ramo da árvore de espaço de busca.

1.4 O Algoritmo de *Bron-Kerbosch*

Para lidar com ambas as tarefas (encontrar o máximo clique e encontrar o menor número de cliques em um grafo), uma abordagem passa por encontrar os cliques maximais em um grafo não direcionado. Dado um grafo não direcionado $G = (V, E)$, um clique C é um subconjunto não vazio de V , em que todos os vértices, tomados dois a dois, são adjacentes em C . Um clique é *maximal* se ele não é subconjunto de nenhum outro clique em G .

O problema de enumerar todos os cliques maximais é NP-difícil e um dos algoritmos mais populares na literatura para resolver esse problema, devido ao seu fácil entendimento e implementação relativamente simples é o algoritmo de *Bron-Kerbosch*.

O algoritmo de *Bron-Kerbosch* utiliza uma estratégia de *backtracking* para resolver o problema de enumerar os cliques maximais de um grafo, dividindo os vértices em diversos subconjuntos determinando, recursivamente, soluções para todos os cliques maximais.

A versão mais simples do algoritmo de *Bron-Kerbosch* faz uso de três conjuntos, R , P e X para buscar e expandir os cliques máximos (Código 3). O conjunto R é o clique maximal atualmente encontrado durante a busca, o conjunto P mantém os vértices a serem explorados, ou seja, os candidatos a serem incluídos no clique maximal atual e o conjunto X guarda os vértices já explorados [5].

```

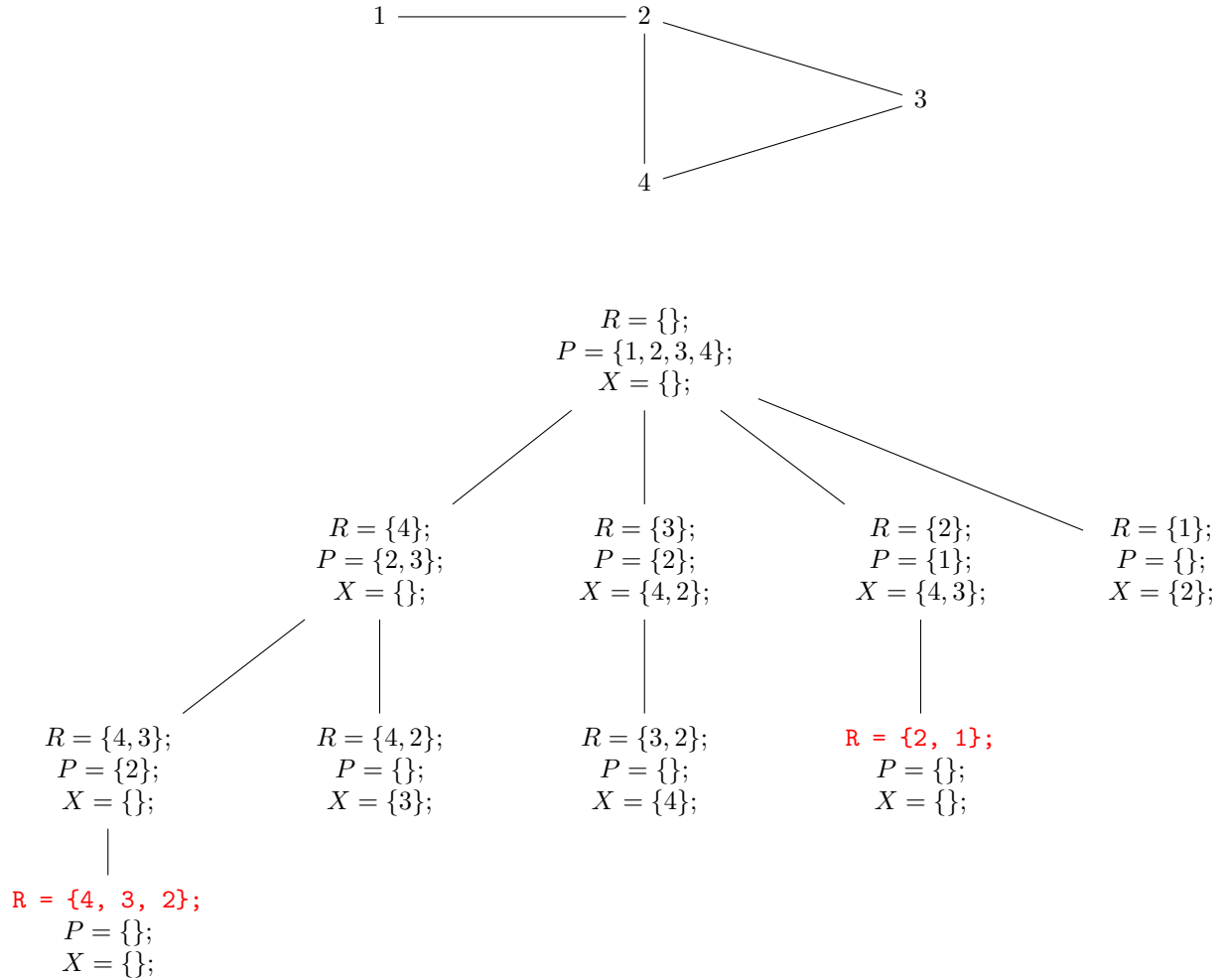
1 // Algorithm 1. Bron-Kerbosch Algorithm
2 // Require: Clique R, candidate set P, and duplicate set X.
3 // Ensure: Identify all maximal cliques.
4 1: procedure BK(R, P, X)
5 2:     if isEmpty(P U X) then
6 3:         report R as a maximal clique
7 4:         return
8 5:     for each vertex v in P do
9 6:         BK(R U {v}, P.intersection(N(v)), X.intersection(N(v)))
10 7:         P = P \ {v}
11 8:         X = X U {v}
12 9: end procedure

```

Código 3: Pseudo-código do Algoritmo de Bron-Kerbosh para o Problema de Enumeração dos Cliques Maximais (Backtracking). Adaptado de [5]

A figura a seguir mostra um exemplo de grafo e sua árvore de recursão gerada pela versão mais simples do algoritmo de *Bron-Kerbosh*. Inicialmente, o algoritmo começa com as seguintes condições: $R = \{\}$, $P = V$ e $X = \{\}$. Como P e X não são ambos vazios, cada vértice v em P é usado para expandir o clique atual. No caso do exemplo, a raiz tem quatro ramificações, porque P possui quatro vértices. A escolha, neste caso, é feita pelo maior vértice em P . A cada passo da recursão, os conjuntos R , P e X são atualizados ($R = R \cup \{v\}$; $P = P \cap N(v)$; $X = X \cup N(v)$) e uma nova chamada recursiva

é realizada, até que ambos P e X sejam vazios. Nessa versão do algoritmo, os cliques maximais encontrados foram $\{4, 3, 2\}$ e $\{2, 1\}$.



1.5 Visão geral do ambiente de desenvolvimento e testes

A seguir, uma visão geral do ambiente de desenvolvimento e testes, com instruções de compilação e uso das soluções implementadas.

1.5.1 Ambiente de desenvolvimento e ferramentas utilizadas

Os algoritmos desenvolvidos para a solução das duas tarefas foram implementados em linguagem *C++* e compilados usando o *gcc 6.3.0* do conjunto de ferramentas MinGW¹. Os programas gerados foram executados em ambiente *Windows 10 Pro - 64 bits*, numa máquina *Intel(R) Core(TM) i5-7500 CPU @ 3.40 GHz 3.41 GHz*, com *16 GB RAM*.

Os programas também foram compilados com o *g++ 5.40 20160609* e executados em ambiente *Linux (Ubuntu 16.04.12 LTS)*, numa máquina *Intel(R) Core(TM) i7-3770 CPU @ 3.40 GHz x8*, com *16 GB RAM*².

1.5.2 Instruções de compilação e execução

Para a compilação do código associado à Tarefa 1, basta digitar, no terminal:

¹<https://sourceforge.net/projects/mingw/files/MinGW/Base/gcc/Version6/gcc-6.3.0/>

²Arquivo de instruções: em <https://acesse.dev/Shucm>

Compilando código para cálculo da multiplicação de matrizes

```
gcc task1-max-clique.cpp -o task1-max-clique
```

Para a execução do programa basta digitar:

```
./task1-max-clique.exe <arquivo.clq | arquivo.grafo> <maxtime> <algoritmo [bp: backtracking | bb: branch&bound]>
```

Para a compilação do código associado à Tarefa 2, basta digitar, no terminal:

Compilando código para o algoritmo de Dijkstra

```
gcc task2-min-cliques.cpp -o task2-min-cliques
```

Para a execução do programa basta digitar:

```
./task2-minimum-number-of-cliques.exe <arquivo.clq | arquivo.grafo> <maxtime>
```

2 Implementação

Nesta seção serão apresentadas as soluções implementadas para o *problema do máximo clique* e para o *problema do mínimo número de cliques*. Para cada solução, descreve-se o funcionamento das principais funções e procedimentos utilizados, bem como detalhes de implementação.

2.1 Implementações para o Problema do Clique Máximo

Uma abordagem para encontrar o clique máximo em um grafo $G = (V, E)$ é enumerar todos os cliques possíveis e selecionar o que contém o maior número de vértices. A complexidade dessa abordagem (força-bruta) é da ordem de $3^{|V|}$. O algoritmo de **Bron-Kerbosch** utiliza uma abordagem recursiva para encontrar todos os cliques e sua ordem de complexidade é de $O(3^{\frac{|V|}{3}})$. Esse algoritmo pode ser usado para obter o clique máximo em G .

O Código 4 apresenta a implementação do algoritmo de **Bron-Kerbosch** [5], versões com estratégia **Backtracking** e **Branch&Bound**, para resolver o problema de encontrar o máximo clique em um grafo não direcionado. Essas variações do algoritmo original de **Bron-Kerbosch** mantêm o controle sobre o clique máximo encontrado em cada chamada recursiva, atualizando-o, quando necessário.

As principais diferenças do **Branch&Bound** em relação ao **Backtracking** são que o primeiro mantém o controle do tamanho do maior clique encontrado, utilizando-o como critério para podar o espaço de busca, além de um pivô, que é utilizado para determinar o próximo vértice a ser escolhido no conjunto de vértices candidatos para a expansão da solução.

```
1  /**
2   * PCC104 - Projeto e Analise de Algoritmos
3   * Departamento de Computacao - Universidade Federal de Ouro Preto - MG
4   * Professor: Pedro Silva
5   * Aluno: Fernando dos Santos Alves Fernandes
6   * Trabalho Pratico II (TP II) - Tarefa 1: Problema do Clique Maximo
7   */
8
9  #include <iostream>
10 #include <fstream>
11 #include <sstream>
12 #include <vector>
13 #include <string>
14 #include <algorithm>
15 #include <chrono>
```

```

16 #include <bits/stdc++.h> //setprecision
17
18 using namespace std;
19
20 class Graph {
21 private:
22     int V;
23     vector<vector<int>> adj;
24     vector<int> maxClique;
25     int maxCliqueSize;
26     double maxTime;
27     long numOfCalls = 0;
28     std::chrono::time_point<std::chrono::_V2::system_clock, std::chrono::
        duration<long long int, std::ratio<111, 100000000011> > > initTime;
29     std::chrono::time_point<std::chrono::_V2::system_clock, std::chrono::
        duration<long long int, std::ratio<111, 100000000011> > > endTime;
30     std::chrono::duration<long long int, std::ratio<111, 100000000011> >
        elapsedTime;
31
32 public:
33     Graph(int V) : V(V) {
34         adj.resize(V);
35     }
36
37     void addEdge(int u, int v) {
38         adj[u].push_back(v);
39         adj[v].push_back(u);
40     }
41
42     // Bron-Kerbosch algorithm (Backtracking version)
43     void bronKerboschBT(vector<int>& R, vector<int>& P, vector<int>& X, Graph&
        g, vector<int>& maxClique) {
44         numOfCalls++;
45         endTime = chrono::high_resolution_clock::now();
46         auto elapsed = chrono::duration_cast<chrono::nanoseconds>(endTime -
            initTime);
47
48         if ((P.empty() && X.empty()) || (elapsed >= chrono::duration_cast<
            chrono::nanoseconds>(std::chrono::duration<double>(maxTime)))) {
49             elapsedTime = elapsed;
50             if (R.size() > maxClique.size()) {
51                 maxClique = R;
52             }
53             return;
54         }
55
56         vector<int> P_copy = P;
57
58         for (int v : P_copy) {
59             vector<int> R_new = R; // R'
60             R_new.push_back(v); // R' = R U {v}
61
62             vector<int> P_new; // P'
63             vector<int> X_new; // X'
64
65             for (int w : P) { // P' = P intersection N(v)
66                 if (find(g.adj[v].begin(), g.adj[v].end(), w) != g.adj[v].end
                    ()) {
67                     P_new.push_back(w);
68                 }
69             }
70

```

```

71         for (int w : X) { //  $X' = X \cap N(v)$ 
72             if (find(g.adj[v].begin(), g.adj[v].end(), w) != g.adj[v].end()
73                 ()) {
74                 X_new.push_back(w);
75             }
76         }
77         bronKerboschBT(R_new, P_new, X_new, g, maxClique); //
78         bronKerboschBT(R ∪ {v}, P ∩ N(v), X ∩ N(v)
79
80         P.erase(find(P.begin(), P.end(), v)); //  $P = P \setminus \{v\}$ 
81         X.push_back(v); //  $X = X \cup \{v\}$ 
82     }
83
84     // Bron-Kerbosch algorithm with pivot and bounding (Branch&Bound version)
85     void bronKerboschBB(vector<int>& R, vector<int>& P, vector<int>& X, Graph&
86         g, vector<int>& maxClique, int& maxCliqueSize) {
87         numOfCalls++;
88         endTime = chrono::high_resolution_clock::now();
89         auto elapsed = chrono::duration_cast<chrono::nanoseconds>(endTime -
90             initTime);
91         //cout << elapsed.count() * 1e-9 << " , " << maxTime << endl;
92
93         if ((P.empty() && X.empty()) || (elapsed >= chrono::duration_cast<
94             chrono::nanoseconds>(std::chrono::duration<double>(maxTime)))) {
95             if (R.size() > maxCliqueSize) {
96                 elapsedTime = elapsed;
97                 maxCliqueSize = R.size();
98                 maxClique = R;
99             }
100             return;
101         }
102
103         int pivot = P.empty() ? -1 : P[0]; // Se P for vazio, nao ha como
104         escolher um pivo.
105         vector<int> P_aux; // Candidatos que nao sao vizinhos de v.
106         for (int v : P) {
107             if (pivot == -1 || find(g.adj[pivot].begin(), g.adj[pivot].end(),
108                 v) == g.adj[pivot].end()) {
109                 P_aux.push_back(v);
110             }
111         }
112
113         for (int v : P_aux) {
114             vector<int> R_new = R;
115             R_new.push_back(v);
116
117             vector<int> P_new;
118             vector<int> X_new;
119
120             for (int w : P) {
121                 if (find(g.adj[v].begin(), g.adj[v].end(), w) != g.adj[v].end()
122                     ()) {
123                     P_new.push_back(w);
124                 }
125             }
126
127             for (int w : X) {
128                 if (find(g.adj[v].begin(), g.adj[v].end(), w) != g.adj[v].end()
129                     ()) {

```

```

123         X_new.push_back(w);
124     }
125 }
126
127     // So prossegue se o tamanho potencial do clique e maior do que o
128     // clique maximo atualmente encontrado.
129     if (R_new.size() + P_new.size() > maxCliqueSize) {
130         bronKerboschBB(R_new, P_new, X_new, g, maxClique,
131             maxCliqueSize);
132     }
133
134     P.erase(find(P.begin(), P.end(), v));
135     X.push_back(v);
136 }
137
138 void setInitialAndMaxTime(std::chrono::time_point<std::chrono::_V2::
139     system_clock, std::chrono::duration<long long int, std::ratio<1ll,
140     1000000000ll> > > initial, double time) {
141     initTime = initial;
142     maxTime = time;
143 }
144
145 auto getElasedTime(){
146     return elapsedTime.count();
147 }
148
149 void printElapsedTime(){
150     cout << "\nTempo de execucao: " << std::fixed << std::setprecision(4)
151         << double (elapsedTime.count() * 1e-9) << " s." << endl;
152 }
153
154 void printMaxClique() {
155     cout << maxCliqueSize << endl;
156
157     for ( int v : maxClique) {
158         cout << v + 1 << " ";
159     }
160     cout << endl;
161 }
162
163 long getNumOfCalls(){
164     return numOfCalls;
165 }
166
167 };
168
169 int main(int argc, char* argv[]) {
170     if (argc != 4) {
171         cerr << "Uso: " << argv[0] << " <arquivoProblema> <tempoMaximo> <
172             estrategia>" << endl;
173         return 1;
174     }
175
176     string filename = argv[1];
177     int timetorun = stoi(argv[2]);
178     string strategy = argv[3];
179
180     ifstream file(filename);
181     if (!file.is_open()) {
182         cerr << "Erro ao abrir o arquivo " << filename << endl;
183         return 1;
184     }

```

```

179
180 string line;
181 unsigned int numVertices = 0;
182 unsigned int numEdges = 0;
183
184 string extension = ".grafo";
185 string::size_type i = filename.find(extension);
186 if (i != string::npos) { // extensao '.grafo'
187     getline(file, line);
188     stringstream ss(line);
189     ss >> numVertices;
190     ss >> numEdges;
191 }else { // extensao '.clq'
192     // Ler o cabecalho do arquivo e obter o numero de vertices e arestas
193     while (getline(file, line)) {
194         if (line.empty() || line[0] == 'p') {
195             stringstream ss(line);
196             string token;
197             ss >> token; // descartar 'p'
198             ss >> token; // esperado 'col'/'edge'
199             ss >> numVertices;
200             ss >> numEdges;
201             break;
202         }
203     }
204 }
205
206 Graph graph(numVertices);
207
208 // Ler as arestas e adicionar ao grafo
209 if (i != string::npos) { // extensao '.grafo'
210     while (getline(file, line)) {
211         if (line.empty()) continue;
212         unsigned int from, to;
213         stringstream ss(line);
214         ss >> from >> to;
215         graph.addEdge(from, to); // Os vertices no arquivo comecam de 0.
216     }
217 } else{
218     while (getline(file, line)) {
219         if (line.empty() || line[0] != 'e') continue;
220         stringstream ss(line);
221         string token;
222         ss >> token; // descartar 'e'
223         unsigned int from, to;
224         ss >> from >> to;
225         graph.addEdge(from - 1, to - 1); // Os vertices no arquivo comecam
           de 1.
226     }
227 }
228 file.close();
229
230 vector<int> R, P, X, maxClique;
231 int maxCliqueSize = 0;
232 for (int i = 0; i < numVertices; ++i) {
233     P.push_back(i);
234 }
235
236 auto initTime = chrono::high_resolution_clock::now();
237 graph.setInitialAndMaxTime(initTime, timetorun);
238 if (strategy == "bp") // Backtracking ou Backpropagation
239     graph.bronKerboschBT(R, P, X, graph, maxClique);

```

```

240     else if (strategy == "bb") // Branch and Bound
241         graph.bronKerboschBB(R, P, X, graph, maxClique, maxCliqueSize);
242     //graph.printMaxClique();
243
244     cout << "Tempo de execucao: " << graph.getElasedTime() * 1e-9 << endl;
245     cout << "Numero de chamadas recursivas: " << graph.getNumOfCalls() << endl
246         ;
247
248     cout << maxClique.size() << endl;
249     for (int v : maxClique) {
250         cout << v + 1 << " "; // Convert back to 1-based index
251     }
252     cout << endl;
253     //graph.printElapsedTime();
254
255     return 0;
}

```

Código 4: Implementação da Tarefa 1 - Problema do Clique Máximo, usando *Backtracking* e *Branch&Bound*.

Para executar o programa com qualquer uma das instâncias de teste, na linha de comando do terminal, conforme descrito na seção 1.5.2, o usuário informa o nome do arquivo de entrada, o tempo máximo de execução (em segundos) e a estratégia que deseja utilizar ('bp' para o *Backtracking* ou 'bb' para o *Branch&Bound*). As saídas são apresentadas na tela, conforme o exemplo de caso de teste da Tabela 1.

2.2 Implementações para o Problema do Mínimo Número de Cliques

O código 5 apresenta a implementação para resolver o problema de encontrar o mínimo número de cliques em um grafo não direcionado. A estratégia utilizada se baseou no uso do algoritmo de *Bron-Kerbosh* (método *findSetOfCliques*) para encontrar todos os cliques maximais e, a partir daí, encontrar a menor cobertura por cliques das arestas do grafo. O método *minimumCliqueCover* percorre a lista de cliques maximais, verificando quais cobrem os vértices do grafo. Ao final dessa verificação, tem-se a melhor cobertura por cliques. Por fim, o conjunto de cliques correspondentes à menor cobertura é ordenado, para a impressão da saída desejada.

```

1  /**
2   * PCC104 - Projeto e Analise de Algoritmos
3   * Departamento de Computacao - Universidade Federal de Ouro Preto - MG
4   * Professor: Pedro Silva
5   * Aluno: Fernando dos Santos Alves Fernandes
6   * Trabalho Pratico II (TP II) - Tarefa 2: Problema do Minimo Numero de
7     Cliques
8   */
9
10 #include <iostream>
11 #include <fstream>
12 #include <sstream>
13 #include <vector>
14 #include <string>
15 #include <limits>
16 #include <chrono>
17 #include <algorithm>
18 #include <bits/stdc++.h> //setprecision
19 #include <unordered_set>
20
21 using namespace std;
22
23 class Graph {
24 private:
25     int V;

```

```

25     vector<vector<int>> > adj;
26     vector<vector<int>> > bestCover; // Solucao
27     int bestCoverSize;
28     double maxTime;
29     long numOfCalls = 0;
30     chrono::time_point<chrono::system_clock> initTime;
31
32 public:
33     Graph(int V) : V(V), bestCoverSize(numeric_limits<int>::max()) {
34         adj.resize(V);
35     }
36
37     void addEdge(int u, int v) {
38         adj[u].push_back(v);
39         adj[v].push_back(u);
40     }
41
42     void setInitialAndMaxTime(chrono::time_point<chrono::system_clock> initial
43         , double time) {
44         initTime = initial;
45         maxTime = time;
46     }
47
48     // Algoritmo de Bron-Kerbosch para encontrar todos os cliques maximais (
49     versao backtracking).
50     void findSetOfCliques(vector<int>& R, vector<int>& P, vector<int>& X,
51         vector<vector<int>> & allCliques) {
52         numOfCalls++;
53         if (P.empty() && X.empty()) {
54             allCliques.push_back(R);
55             return;
56         }
57
58         while (!P.empty()) {
59             int v = P.back();
60             P.pop_back();
61             vector<int> R_new = R;
62             R_new.push_back(v);
63
64             vector<int> P_new;
65             vector<int> X_new;
66             for (int w : P) {
67                 if (find(adj[v].begin(), adj[v].end(), w) != adj[v].end()) {
68                     P_new.push_back(w);
69                 }
70             }
71             for (int w : X) {
72                 if (find(adj[v].begin(), adj[v].end(), w) != adj[v].end()) {
73                     X_new.push_back(w);
74                 }
75             }
76
77             findSetOfCliques(R_new, P_new, X_new, allCliques);
78             X.push_back(v);
79
80             // Limite de tempo de execucao
81             if (chrono::duration_cast<chrono::seconds>(chrono::system_clock::
82                 now() - initTime).count() > maxTime) {
83                 return;
84             }
85         }
86     }
87 }

```

```

83
84     vector<vector<int>> > cliqueCover() {
85         vector<vector<int>> > allCliques;
86         vector<int> R, P(V), X;
87         iota(P.begin(), P.end(), 0); // Inicializa P (conjunto de vertices
            candidatos) com todos os vertices.
88
89         findSetOfCliques(R, P, X, allCliques);
90         return allCliques;
91     }
92
93     bool compareSize(vector<int> clique1, vector<int> clique2) {
94         return (clique1.size() > clique2.size());
95     }
96
97     void printSolution() {
98         // Ordena os cliques pelo tamanho, do maior para o menor.
99         sort(bestCover.begin(), bestCover.end(), [](const vector<int>& a,
            const vector<int>& b) {
100             return a.size() > b.size();
101         });
102
103         cout << bestCover.size() << endl;
104         for (const auto& clique : bestCover) {
105             for (int v : clique) {
106                 cout << v + 1 << " ";
107             }
108             cout << "\n";
109         }
110     }
111
112     void minimumCliqueCover() {
113         auto allCliques = cliqueCover();
114         vector<int> covered(V, 0);
115         vector<vector<int>> > min_cover;
116
117         for (const auto& clique : allCliques) { // Para cada clique, checa se
            cobre algum vertice ainda nao coberto.
118             bool is_covered = false;
119             for (int v : clique) {
120                 if (!covered[v]) {
121                     is_covered = true;
122                     break;
123                 }
124             }
125             if (is_covered) { // Se pelo menos um vertice ainda nao foi
                coberto, todos os vertices do clique passam a ser cobertos.
126                 min_cover.push_back(clique); // O clique e incluido na
                    cobertura.
127                 for (int v : clique) {
128                     covered[v] = 1;
129                 }
130             }
131         }
132
133         bestCover = min_cover;
134     }
135
136     long getNumOfCalls(){
137         return numOfCalls;
138     }
139 };

```

```

140
141 int main(int argc, char* argv[]) {
142     if (argc != 3) {
143         cerr << "Uso: " << argv[0] << " <arquivoProblema> <tempoMaximo>" <<
            endl;
144         return 1;
145     }
146
147     string filename = argv[1];
148     double timetorun = stod(argv[2]);
149
150     ifstream file(filename);
151     if (!file.is_open()) {
152         cerr << "Erro ao abrir o arquivo " << filename << endl;
153         return 1;
154     }
155
156     string line;
157     unsigned int numVertices = 0;
158     unsigned int numEdges = 0;
159
160     string extension = ".grafo";
161     string::size_type i = filename.find(extension);
162     if (i != string::npos) {
163         getline(file, line);
164         stringstream ss(line);
165         ss >> numVertices;
166         ss >> numEdges;
167     } else {
168         while (getline(file, line)) {
169             if (line.empty() || line[0] == 'p') {
170                 stringstream ss(line);
171                 string token;
172                 ss >> token; // descartar 'p'
173                 ss >> token; // esperado 'col'/'edge'
174                 ss >> numVertices;
175                 ss >> numEdges;
176                 break;
177             }
178         }
179     }
180
181     Graph graph(numVertices);
182
183     if (i != string::npos) {
184         while (getline(file, line)) {
185             if (line.empty()) continue;
186             unsigned int from, to;
187             stringstream ss(line);
188             ss >> from >> to;
189             graph.addEdge(from, to);
190         }
191     } else {
192         while (getline(file, line)) {
193             if (line.empty() || line[0] != 'e') continue;
194             stringstream ss(line);
195             string token;
196             ss >> token; // descartar 'e'
197             unsigned int from, to;
198             ss >> from >> to;
199             graph.addEdge(from - 1, to - 1);
200         }

```

```

201     }
202     file.close();
203
204     auto startTime = chrono::system_clock::now();
205     graph.setInitialAndMaxTime(startTime, timetorun);
206
207     graph.minimumCliqueCover();
208     //cout << "Numero de chamadas recursivas: " << graph.getNumOfCalls() <<
209         endl;
210     graph.printSolution();
211     return 0;
212 }

```

Código 5: Implementação da Tarefa 2 - Problema do Mínimo Número de Cliques, com *Backtracking*.

Conforme as instruções de uso (seção 1.5.2), na execução do programa desenvolvido, o usuário passa o nome do arquivo de instância e o tempo máximo de execução até a apresentação de uma saída. Para essa tarefa, apenas a versão com *Backtracking* foi implementada, portanto, não é necessário passar o método desejado na linha de comando.

A função principal faz a leitura do arquivo de entrada, ignorando as linhas de comentário (iniciadas com o caractere 'c'), e obtém o número total de vértices e número total de arestas. Em seguida, cria um objeto do tipo *Graph*, e continua a leitura do arquivo de entrada, atribuindo ao grafo, para cada linha lida do arquivo, uma nova aresta.

A partir daí, inicia-se a contagem do tempo limite de execução do programa, o método do grafo para obter o menor número de cliques é chamado, seguido pelo método de impressão.

3 Estudo de Complexidade

Nesta seção são apresentados os estudos da complexidade do tempo de execução dos procedimentos implementados e do programa como um todo, considerando entradas de tamanho n .

3.1 Análise de complexidade para o Problema do Clique Máximo

A estratégia para resolver o problema de encontrar o clique máximo em um grafo não direcionado $G = (V, E)$ é baseada na obtenção de todos os cliques máximos de G . Segundo [4], um grafo com n vértices tem, no máximo, $3^{\frac{n}{3}}$ cliques maximais. Portanto, no pior caso, em que o algoritmo de Bron-Kerbosch precisa explorar todas as combinações possíveis de subconjuntos de vértices para encontrar os cliques maximais, tem ordem de complexidade $O(3^{\frac{n}{3}})$.

3.2 Análise de complexidade para o Problema do Mínimo Número de Cliques

Analogamente, resolver o problema de encontrar o mínimo número de cliques em um grafo não direcionado $G = (V, E)$ passa por encontrar a cobertura mínima de G por cliques. Mais uma vez, a estratégia baseia-se na etapa de obtenção dos cliques maximais, usando o algoritmo de Bron-Kerbosch, somada uma verificação de que os cliques encontrados cobrem os vértices ou as arestas do grafo G . Portanto, no pior caso, a ordem e complexidade da tarefa é de $O(3^{\frac{n}{3}})$.

4 Testes

Esta seção descreve os experimentos realizados para os dois problemas abordados neste trabalho prático: **Tarefa 1 (Clique Máximo)** e **Tarefa 2 (Menor Número de Cliques)**. Para ambos os problemas, foram usados os mesmos arquivos de instância, passados como parâmetro via terminal.

A saída da **Tarefa 1** corresponde ao tamanho do clique máximo encontrado, seguido pelos vértices que o compõe, conforme o exemplo na Tabela 1. Para a **Tarefa 2**, a saída é o número de subcliques encontrados, seguido por cada um deles na linha abaixo, do maior para o menor, como o caso de teste da Tabela 2.

O objetivo principal dos testes realizados foi avaliar se as soluções atendiam às especificações de cada tarefa, em termos de ser capaz de entregar corretamente, para cada instância de entrada, a saída desejada. A execução dos programas é limitada com o auxílio da biblioteca [<chrono>](#).

4.1 Testes para o Problema do Clique Máximo

Durante a implementação da solução para o problema do clique máximo, algumas instâncias de teste foram geradas, de acordo com o formato do *Exemplo de caso de teste* (*brock200_4.clq*) apresentado na especificação da **Tarefa 1** (Tabela 1).

Entrada	Saída
c FILE: brock200_4.b	17
p edge 200 13089	160 27 116 164 126 53 64 78 191 138 11 92 37 70 185 18 28
e 3 1	
e 4 3	
e 5 1	
e 5 3	
e 6 1	
e 6 2	
e 6 3	
e 6 5	
e 7 1	
...	

Tabela 1: Exemplo de entrada e saída esperada para a **Tarefa 1**.

Para o problema do clique máximo foram implementadas duas versões do algoritmo de *Bron-Kerbosch*, uma versão *Backtracking* (sem pivô) e uma versão *Branch&Bound* (com pivô).

A versão com pivô escolhe vértice u a ser adicionado no clique entre vértices candidatos que não são adjacentes ao vértice v , atualmente sendo explorado. A ideia é considerar uma cobertura por vértices, dessa forma evitam-se cliques maximais com vértices já pertencentes a cliques já encontrados. Essa estratégia de poda se reflete na redução do número de chamadas recursivas e, portanto, na redução da ramificação da árvore de busca.

A seguir, temos as saídas para as principais instâncias de teste, considerando um limite de tempo de execução igual a 1 minuto (60 segundos) e as duas versões do *Bron-Kerbosch* (bp: *Backtracking*, bb: *Branch&Bound*).

brock200_4.clq:

```
>task1-max-clique.exe brock200_4.clq 60 bp
15
1 3 6 22 23 31 33 40 53 72 85 90 95 98 124

>task1-max-clique.exe brock200_4.clq 60 bb
15
1 3 11 8 20 31 33 40 53 72 76 90 91 95 193
```

c125.9.clq:

```
>task1-max-clique.exe c125.9.clq 60 bp
29
1 2 4 5 6 7 8 9 10 11 17 18 19 21 35 39 41 45 63 73 80 89 91 98 99 100 103 107 119

>task1-max-clique.exe c125.9.clq 60 bb
31
1 2 4 5 6 7 40 9 29 11 17 18 19 24 79 82 41 44 45 54 100 80 98 101 103 107 110 115 117 121 122
```

C250.9.clq:

```
>task1-max-clique.exe C250.9.clq 60 bp
32
1 2 5 6 8 12 13 14 21 23 29 30 32 36 41 50 55 63 66 77 94 99 115 118 142 160 197 201 218 234
242 248

>task1-max-clique.exe C250.9.clq 60 bb
35
1 2 5 6 8 12 13 14 21 23 29 30 33 36 41 49 50 55 66 63 77 94 99 115 118 136 147 160 181 197
201 218 234 242 248
```

d8f1b2660f576d7d.grafo:

```
>task1-max-clique.exe d8f1b2660f576d7d.grafo 60 bp
22
1 2 3 4 7 12 16 18 35 43 61 63 109 141 158 209 245 423 648 682 747 945

>task1-max-clique.exe d8f1b2660f576d7d.grafo 60 bb
24
1 2 3 4 7 12 16 55 43 61 63 562 141 875 161 217 398 573 648 682 803 865 932 977
```

4.2 Testes para o Problema do Mínimo Número de Cliques

As instâncias de teste para o problema de encontrar o mínimo número de cliques em grafos não direcionados foram as mesmas da tarefa 1. O *Exemplo de caso de teste*, para a instância *C125.9.clq*, pode ser visto na Tabela 2.

Entrada	Saída
c FILE: C125.9.clq	7 // Parte da entrada foi suprimida para caber em uma linha
p col 125 6963	24 15 18 21 29 31 35 44 49 54 64 68 72 74 82 83 92 ...
e 2 1	23 1 2 10 11 27 32 46 58 60 61 73 76 78 81 85 88 94 ...
e 3 1	21 4 12 14 16 20 26 39 45 50 51 55 57 62 65 66 75 86 ...
e 4 1	22 6 8 9 13 19 23 30 33 37 38 42 47 52 53 56 59 80 87 ...
e 4 2	18 3 7 28 40 41 63 67 69 71 84 91 93 97 98 99 104 106 ...
e 4 3	14 5 17 24 25 34 48 70 77 79 89 115 122 123 125
e 5 1	3 22 43 96
e 5 2	
e 5 3	
e 5 4	
...	

Tabela 2: Exemplo de entrada e saída para a **Tarefa 2**.

A seguir, temos as saídas para as principais instâncias de teste, considerando um limite de tempo de execução igual a 1 minuto (60 segundos). Parte dos resultados foi suprimida em função do tamanho da saída.

brock200_4.clq:

```
>task1-max-clique.exe brock200_4.clq 60
97
200 199 198 193 192 170 157 133 108 99 98 35 33 9
200 199 198 193 192 170 160 157 133 99 58 33 28 9
200 199 198 193 192 170 160 157 133 108 99 58 33 9
200 198 193 191 177 170 163 162 125 98 28 15 9
200 199 198 195 192 170 160 157 151 139 119 108 95
200 198 196 193 180 159 157 148 143 132 79 42 36
200 198 193 192 191 177 170 166 160 133 89 28 9
200 199 198 196 195 173 167 127 80 49 39 36
200 199 198 193 182 180 170 167 152 58 33 28
200 198 193 192 191 177 170 166 160 133 97 9
200 198 196 195 191 180 141 139 76 70 65 25
200 198 196 195 191 180 147 139 138 76 70 25
200 198 196 193 180 170 157 143 99 79 58 20
200 199 198 196 193 167 160 133 99 80 62 11
200 199 198 196 193 167 160 148 133 71 68 36
200 198 195 185 180 167 139 114 109 102 80 36
200 198 196 195 191 180 153 139 138 48 25 19
200 198 196 195 191 180 159 141 139 114 25 19
200 198 195 192 179 173 159 127 123 114 35 16
200 199 198 196 195 170 167 160 139 97 76 11
200 199 198 196 195 170 167 160 139 97 76 19
200 199 198 196 195 173 127 115 49 46 36 19
200 199 195 185 180 179 167 139 92 80 36 7
200 199 196 193 190 180 167 99 58 25 18
200 199 198 196 195 180 170 167 139 76 19
...
```

c125.9.clq:

```
>task1-max-clique.exe c125.9.clq 60
27
125 124 123 122 121 120 119 118 114 111 110 108 104 103 99 80 74 70 62 60 56 52 50 49 29 25
125 124 123 122 121 120 119 118 114 112 111 110 108 104 103 99 80 74 60 56 52 50 49 29 25
125 124 123 122 121 120 119 118 114 112 111 110 108 104 103 99 80 74 60 56 52 50 49 29 4
125 124 123 122 121 120 119 118 114 112 111 110 108 104 99 93 80 74 71 60 49 48 29 26 25
125 124 123 122 121 120 119 118 114 112 111 110 108 104 93 80 74 71 60 49 48 47 29 26 25
125 124 123 122 121 120 119 118 114 112 111 110 104 103 99 93 80 63 60 52 49 30 29 23 20
125 124 123 122 121 120 119 118 114 112 111 110 104 103 99 80 74 60 56 52 50 49 29 25 14
125 124 123 122 121 120 119 118 114 112 111 110 104 103 99 93 80 63 60 52 49 30 29 5
125 124 123 122 121 120 119 118 114 111 110 103 101 93 82 80 74 70 60 52 49 38 30 29
125 124 123 122 121 120 119 118 114 111 110 104 103 99 93 80 74 70 60 52 49 30 29 28
...
```

C250.9.clq:

```

>task1-max-clique.exe C250.9.clq 60
20
250 248 247 246 245 244 243 240 239 233 232 231 228 227 210 209 206 204 189 184 180 171 151
147 141 125 124 94 77 50 11
250 248 247 246 245 244 243 240 239 233 232 231 228 227 210 209 206 204 189 186 184 180 171
151 147 143 141 94 77 50 11
250 248 247 246 245 244 243 240 239 233 232 231 228 227 210 209 206 204 189 184 171 153 151
147 143 125 94 88 38 11
250 248 247 246 245 244 243 240 239 233 232 231 228 227 210 209 206 204 189 186 184 180 171
153 151 147 143 94 77 11
250 248 247 246 245 244 243 240 239 233 232 231 228 227 210 209 206 204 189 186 184 180 171
153 151 147 143 94 88 11
250 248 247 246 245 244 243 240 239 233 232 231 228 227 210 209 207 206 189 184 180 171 164
153 147 125 87 74 66 18
250 248 247 246 245 244 243 240 239 233 232 231 228 227 210 209 207 206 189 186 184 180 171
164 153 151 147 143 88 87
250 248 247 246 245 244 243 240 239 233 232 231 228 227 210 209 207 206 204 189 184 180 171
153 151 147 143 125 88 68
250 248 247 246 245 244 243 240 239 233 232 231 228 227 210 209 207 206 204 186 184 180 171
153 151 147 143 139 88
250 248 247 246 245 244 243 240 239 233 232 231 228 227 210 209 207 204 189 186 184 180 171
153 151 147 143 88 3
250 248 247 246 245 244 243 240 239 233 232 231 228 227 210 209 207 206 204 186 180 171 153
151 147 143 139 99 88
250 248 247 246 245 244 243 240 239 233 232 231 228 227 210 209 207 206 204 189 186 184 180
171 153 151 147 143 88
250 248 247 246 245 244 243 240 239 233 232 231 228 227 210 209 207 206 204 189 186 180 171
151 147 143 141 88 57
250 248 247 246 245 244 243 240 239 233 232 231 228 227 210 209 207 206 204 189 186 184 180
159 153 151 143 88 32
250 248 247 246 245 244 243 240 239 233 232 231 228 227 210 209 207 206 204 189 186 184 180
171 151 147 143 141 88
250 248 247 246 245 244 243 240 239 233 232 231 228 227 210 209 207 206 204 189 186 184 180
171 153 147 143 88 66
250 248 247 246 245 244 243 240 239 233 232 231 228 227 210 209 207 206 204 189 186 184 180
171 153 151 47 32 18
...

```

d8f1b2660f576d7d.grafo:

```

>task1-max-clique.exe d8f1b2660f576d7d.grafo 60 bb
52
1000 999 996 990 987 974 966 963 948 895 879 874 862 766 712 605 545 540 403 400 389 316
237
1000 999 996 990 987 974 966 963 948 895 879 874 862 766 712 605 545 540 472 400 389 237
1000 999 996 990 987 974 966 963 948 895 879 874 766 723 712 605 545 540 472 400 389 237
1000 999 996 990 987 974 966 963 948 895 874 766 723 712 605 545 543 540 472 427 389
1000 999 996 990 987 974 966 963 948 895 879 874 862 766 712 605 545 540 472 400 211
1000 999 996 990 987 974 966 963 948 879 878 874 862 766 712 545 540 472 400 389 237
1000 999 996 990 987 974 966 963 948 879 878 874 797 766 723 712 545 540 472 400 389
1000 999 996 990 987 974 966 963 948 879 878 874 797 766 712 545 540 472 418 400 389
1000 999 996 990 987 974 966 963 948 879 878 874 797 723 712 648 540 472 400 389 71
1000 999 996 990 987 974 966 963 948 879 878 862 766 712 545 540 472 400 391 389 237
1000 999 996 990 987 974 966 963 948 879 878 862 766 610 545 479 472 391 389 373 318
1000 999 996 990 987 974 966 963 895 879 874 862 766 610 605 545 540 472 389 237 224
1000 999 996 990 987 974 966 963 895 874 862 766 712 605 545 540 535 400 389 316 237
1000 999 996 990 987 974 966 963 884 882 878 804 797 766 723 712 686 540 400 316 295
1000 999 996 990 987 974 966 963 884 882 878 804 797 763 723 712 648 540 472 400 71
1000 999 996 990 987 974 966 963 884 882 878 804 766 723 712 686 660 400 316 295 102
1000 999 996 990 987 974 966 963 884 882 878 797 766 744 727 723 712 540 472 400 211
1000 999 996 990 987 974 966 963 948 895 879 874 862 766 712 605 529 400 389 316 237
1000 999 996 990 987 974 966 963 948 895 879 874 862 766 712 545 472 400 389 318 237
1000 999 996 990 987 974 966 963 948 895 879 874 862 766 610 605 545 540 472 389 237
...

```

5 Análises e Considerações

A Tabela 3 apresenta alguns números relacionados à execução das instâncias de teste com as duas versões implementadas para o *Problema do Clique Máximo*.

Observa-se que, para um dado limite de tempo de execução (60 segundos), para a maioria das instâncias das instâncias, a versão do algoritmo de *Bron-Kerbosch* com pivô conseguiu obter um clique máximo com mais vértices do que a versão sem pivô. Além disso, a versão com pivô realiza, média entre 10% a 15% das chamadas recursivas que a versão sem pivô realiza.

Tabela 3: Resultados dos testes para o problema do clique máximo.

instância	# vértices	# arestas	# ch. recursivas	# vért. m. c.
brock200_4.clq (bp)	200	13089	6120939	15
brock200_4.clq (bb)	200	13089	629183	15
c125.9.clq (bp)	125	6963	6612635	29
c125.9.clq (bb)	125	6963	878156	31
C250.9.clq (bp)	250	27984	4195774	32
C250.9.clq (bb)	250	27984	662884	35
d8f1b2660f576d7d.grafo (bp)	1000	369662	1684553	22
d8f1b2660f576d7d.grafo (bb)	1000	369662	178868	24

Os testes com a implementação para o *Problema do Número Mínimo de Cliques* mostraram um número de cliques encontrados bem maior do que o esperado. Por exemplo, para a instância *brock200_4.clq* esperava-se um total de 22 cliques, mas foram encontrados 97; para a instância *c125.9.clq*, esperava-se um total de 7 cliques, mas foram encontrados 27. A razão disso pode ser por conta do tempo limite estabelecido, que não permite que os cliques maximais encontrados (até o momento) sejam expandidos até os realmente maximais em G . Portanto, a cobertura, de vértices ou de arestas, têm que ser realizada por um maior número de cliques.

Referências

- [1] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms - Third Edition*. The MIT Press, Cambridge, Massachusetts, 2009.
- [2] S. Dasgupta, C. H. Papadimitriou, and U. V. Vazirani. *Algorithms*. Wiley, 2006.
- [3] Donald L. Kreher and Douglas R. Stinson. *Combinatorial algorithms: generation, enumeration, and search*. CRC Press LLC, Boca Raton, Florida, 1999.
- [4] J. W. Moon and L. Moser. On cliques in graphs. *Israel J. Math*, 3:23–28, 1965.
- [5] Yi-Wen Wei, Wei-Mei Chen, and Hsin-Hung Tsai. Accelerating the bron-kerbosh algorithm for maximal clique enumeration using gpus. *IEEE Transactions on Parallel and Distributed Systems*, 32(9):2352–2366, 2021.